

Assignment Submission

Solving Linear Programming and Transportation Problems in Python

Name	Meer Kapadia
Roll No.	2503CS02
Course	CS2105 : Optimization Technique
Date	13 September 2026

This report covers two Python programs written for the Optimization Techniques assignment: (1) a general Big-M Simplex solver that handles $\leq$, $\geq$ and $=$ constraints using symbolic-M arithmetic, and (2) a general Transportation Problem solver that builds an initial feasible solution with Vogel's Approximation Method (VAM) and then optimizes it with the Modified Distribution (MODI) method. Both programs take problem data interactively from the user rather than hard-coding it, so they work for any problem size.

Assignment 1 — Big-M Simplex Method

1.1 Objective

To write a Python program that solves a Linear Programming Problem (LPP) using the Big-M method, for a Maximization or Minimization objective, with any mix of $\leq$, $\geq$ and $=$ constraints, and any number of decision variables.

1.2 Theory in brief

The Big-M method converts an LPP into standard form by adding slack variables for $\leq$ constraints, surplus and artificial variables for $\geq$ constraints, and an artificial variable for $=$ constraints. Each artificial variable is penalized in the objective function by a very large constant M , so the simplex algorithm is forced to drive every artificial variable out of the basis (to zero) if a feasible solution exists. Because M is symbolic rather than a fixed large number, every cost and every Dev.Row (net evaluation row) entry is kept as a pair (constant, coefficient of M), and comparisons between two such entries first compare their M -coefficients and only fall back to the constants when the M -coefficients are equal. The optimal solution is reached once no column has a positive Dev.Row entry; if any artificial variable remains in the basis at a positive value at that point, the original problem is infeasible.

1.3 Program design

get_problem_from_user() reads the number of variables and constraints, the optimization sense, the objective coefficients, and each constraint's coefficients, relation and RHS.

build_standard_form() flips any constraint with a negative RHS, negates the objective internally for a minimization problem (so the same maximize-style simplex loop always applies), and appends slack / surplus / artificial columns as needed, using (constant, M -coefficient) tuples for every cost entry.

solve_big_m() runs the simplex iterations: it prints each tableau, picks the entering column as the one with the lexicographically largest Dev.Row value (M -coefficient first, then constant), applies the minimum-ratio test to find the leaving row, and pivots. It stops when no Dev.Row entry is positive, flags an unbounded problem if every entry in the entering column is ≤ 0 , and caps the run at 50 iterations as a safety net against cycling.

report_solution() checks whether any artificial variable is still basic and positive (infeasible case) and otherwise prints the decision variable values and the optimal Z , computed from the original (un-negated) objective coefficients.

1.4 Source cod

```
1 import numpy as np
2
3 TOL = 1e-9
4
5 def madd(a, b):
6     return (a[0] + b[0], a[1] + b[1])
7
8
9 def msub(a, b):
10    return (a[0] - b[0], a[1] - b[1])
11
12
13 def mscale(a, k):
14    return (a[0] * k, a[1] * k)
15
16
17 def m_is_positive(a):
18    if abs(a[1]) > TOL:
19        return a[1] > 0
20    return a[0] > TOL
21
22
23 def format_m(a):
24    const, coeff = a
25    const = round(const, 4)
26    coeff = round(coeff, 4)
27
28    if abs(coeff) < TOL:
29        return f"{const:g}"
30
31    coeff_str = "" if abs(coeff - 1) < TOL else ("+" if abs(coeff + 1) < TOL else f"{coeff:g}")
32    m_part = f"{coeff_str}M" if coeff_str != "+" else "-M"
33
34    if abs(const) < TOL:
35        return m_part
36
37    if const > 0:
38        return f"({m_part}+{const:g})" if not m_part.startswith("-") else f"{const:g}{m_part}"
39    else:
40        return f"({m_part}-{const:g})" if not m_part.startswith("-") else f"({const:g}{m_part}"
41
42
43 def get_problem_from_user():
44    print("\n== Big-M Simplex Solver (symbolic M) ==\n")
45    n = int(input("How many decision variables (x1, x2, ...)? "))
46    m = int(input("How many constraints? "))
47    sense = input("Minimize or Maximize? (type min/max): ").strip().lower()
48
49    print(f"\nEnter the {n} objective function coefficients, space separated")
50    print(f"(example: for Z = 4x1 + x2 just type: 4 1)")
51    obj = list(map(float, input("Objective coefficients: ").split()))
52
53    constraints = []
54    print("\nNow enter each constraint one by one.")
55    print("(Don't type the plain x_i >= 0 lines - those are assumed automatically.)")
56    for i in range(m):
57        print(f"\n-- Constraint {i + 1} --")
58        coeffs = list(map(float, input(f"Coefficients of x1..x{n}: ").split()))
59        relation = input("Relation type <=, >=, or =: ").strip()
60        rhs = float(input("RHS value: "))
61        constraints.append((coeffs, relation, rhs))
62
63    return n, m, sense, obj, constraints
64
65
66 def build_standard_form(n, m, sense, obj, constraints):
67
68    col_names = [f"x{i+1}" for i in range(n)]
69    rows = [list(c[0]) for c in constraints]
70    rhs = [c[2] for c in constraints]
71    relations = [c[1] for c in constraints]
72
73    for i in range(m):
74        if relations[i] <="":
75            rows[i] = [-v for v in rows[i]]
76            rhs[i] = -rhs[i]
77            if relations[i] == "<=":
78                relations[i] = ">="
79            elif relations[i] == ">=":
80                relations[i] = "<="
81
82    if sense.startswith("min"):
83        internal_obj = [-c for c in obj]
84    else:
85        internal_obj = list(obj)
86
87    cost = [(c, 0.0) for c in internal_obj]
88    basis = [None] * m
89
90    for i in range(m):
91        if relations[i] == "<=":
92            col_names.append(f"s{i+1}")
93            col = [0.0] * m
94            col[i] = 1.0
95            cost.append((0.0, 0.0))
96            for r in range(m):
97                rows[r].append(col[r])
98            basis[i] = len(col_names) - 1
99
100        elif relations[i] == ">=":
101            col_names.append(f"s{i+1}")
102            col = [0.0] * m
103            col[i] = -1.0
104            cost.append((0.0, 0.0))
105            for r in range(m):
106                rows[r].append(col[r])
107
108            col_names.append(f"a{i+1}")
109            col2 = [0.0] * m
110            col2[i] = 1.0
111            cost.append((0.0, -1.0))
112            for r in range(m):
113                rows[r].append(col2[r])
114            basis[i] = len(col_names) - 1
115
116        elif relations[i] == "=":
117            col_names.append(f"a{i+1}")
118            col = [0.0] * m
119            col[i] = 1.0
120            cost.append((0.0, -1.0))
121            for r in range(m):
122                rows[r].append(col[r])
123            basis[i] = len(col_names) - 1
124
125    else:
126        raise ValueError(f'Relation '{relations[i]}' not recognized, use <=, >= or =')
127
128    A = np.array(rows, dtype=float)
129    b = np.array(rhs, dtype=float)
130
131    return A, b, cost, basis, col_names, sense
132
133
```

```

134 COL_W = 11
135 LABEL_W = 8
136
137
138 def _border(num_cols):
139     return "+" * (LABEL_W + 2) + ("+" + "-" * (COL_W + 2)) * num_cols + "+"
140
141
142 def _row(label, values):
143     cells = ""
144     for v in values:
145         cells += f"{v:~{COL_W}} |"
146     return "|" + f"{label:~{LABEL_W}} |" + cells
147
148 def print_table(A, b, cost, basis, col_names, iteration):
149     print(f"----- Table {iteration} -----")
150     num_cols = len(col_names) + 1 # +1 for the RHS column
151     border = _border(num_cols)
152     print(border)
153     print(_row("Basis", col_names + ["RHS"]))
154     print(border)
155     for i, var in enumerate(basis):
156         values = [f"{A[i, j]:.3f}" for j in range(len(col_names))] + [f"{b[i]:.3f}"]
157         print(_row(col_names[var], values))
158     print(border)
159
160     cb = [cost[v] for v in basis]
161
162     dev_row = []
163     for j in range(len(col_names)):
164         zj = (0.0, 0.0)
165         for i in range(len(basis)):
166             zj = madd(zj, b[i], A[i, j])
167             dev_row.append(msub(cost[j], zj))
168
169     z_val = (0.0, 0.0)
170     for i in range(len(basis)):
171         z_val = madd(z_val, b[i], cb[i])
172
173     dev_values = [format_m(d) for d in dev_row] + [f"z={format_m(z_val)}"]
174     print(_row("Dev.Row", dev_values))
175     print(border)
176     return dev_row
177
178
179 def solve_big_m(A, b, cost, basis, col_names):
180     iteration = 0
181     while True:
182         iteration += 1
183         dev_row = print_table(A, b, cost, basis, col_names, iteration)
184
185         if not any(m_is_positive(d) for d in dev_row):
186             print("\nNo positive entries left in the Dev.Row -> optimal solution reached.")
187             break
188
189         # entering column = the one with the largest Dev.Row value
190         entering = max(range(len(dev_row)), key=lambda j: (dev_row[j][1], dev_row[j][0]))
191
192         ratios = []
193         for i in range(len(b)):
194             if A[i, entering] > TOL:
195                 ratios.append(b[i] / A[i, entering])
196             else:
197                 ratios.append(np.inf)
198
199         if all(r == np.inf for r in ratios):
200             print("\nAll entries in the pivot column are <= 0 -> problem is UNBOUNDED.")
201             return None, None, basis
202
203         leaving_row = int(np.argmin(ratios))
204         print(f"Entering variable: {col_names[entering]} Leaving variable: {col_names[basis[leaving_row]]}")
205
206         pivot_val = A[leaving_row, entering]
207         A[leaving_row, :] /= pivot_val
208         b[leaving_row] /= pivot_val
209         for i in range(len(b)):
210             if i != leaving_row and abs(A[i, entering]) > 1e-12:
211                 factor = A[i, entering]
212                 A[i, :] -= factor * A[leaving_row, :]
213                 b[i] -= factor * b[leaving_row]
214
215         basis[leaving_row] = entering
216
217         if iteration > 50:
218             print("Stopped after 50 iterations (something is probably cycling).")
219             break
220
221     return A, b, basis
222
223
224 def report_solution(n, sense, obj, col_names, basis, b):
225     for i, var in enumerate(basis):
226         if col_names[var].startswith("a") and b[i] > 1e-6:
227             print(f"\nOriginal problem is INFEASIBLE "
228                   "(an artificial variable could not be driven to zero).")
229             return
230
231     values = {name: 0.0 for name in col_names}
232     for i, var in enumerate(basis):
233         values[col_names[var]] = round(b[i], 4)
234
235     print(f"----- Final Answer -----")
236     x_values = []
237     for i in range(n):
238         xi = values[f"x{i+1}"]
239         x_values.append(xi)
240         print(f"x{i+1} = {xi}")
241
242     original_Z = sum(obj[i] * x_values[i] for i in range(n))
243
244     label = "minimum" if sense.startswith("min") else "maximum"
245     print(f"Optimal Z ({label}) = {round(original_Z, 4)}")
246
247
248 if __name__ == "__main__":
249     n, m, sense, obj, constraints = get_problem_from_user()
250     A, b, cost, basis, col_names, sense = build_standard_form(n, m, sense, obj, constraints)
251     A, b, basis = solve_big_m(A, b, cost, basis, col_names)
252     if A is not None:
253         report_solution(n, sense, obj, col_names, basis, b)
254
255

```

1.5 Sample run

The program was tested on the following mixed-constraint minimization problem, which needs one equality, one $\geq$, and one $\leq$ constraint — a good check since it exercises artificial variables, a surplus variable, and a slack variable all at once:

```
Minimize    Z = 4x1 + x2
subject to  3x1 + x2 = 3
            4x1 + 3x2 >= 6
            x1 + 2x2 <= 4
            x1, x2 >= 0
```

Console input/output :

```
=== Big-M Simplex Solver (symbolic M) ===
How many decision variables (x1, x2, ...)? 2
How many constraints? 3
Minimize or Maximize? (type min/max): min

Enter the 2 objective function coefficients, space separated
(example: for Z = 4x1 + x2 just type: 4 1)
Objective coefficients: 4 1

Now enter each constraint one by one.
(Don't type the plain x_i >= 0 lines - those are assumed automatically.)

-- Constraint 1 --
Coefficients of x1..x2: 3 1
Relation (type <=, >=, or =): =
RHS value: 3

-- Constraint 2 --
Coefficients of x1..x2: 4 3
Relation (type <=, >=, or =): >=
RHS value: 6

-- Constraint 3 --
Coefficients of x1..x2: 1 2
Relation (type <=, >=, or =): <=
RHS value: 4

--- Table 1 ---
| Basis | x1 | x2 | a1 | s2 | a2 | s3 | RHS |
|---|---|---|---|---|---|---|---|
| a1 | 3.000 | 1.000 | 1.000 | 0.000 | 0.000 | 0.000 | 3.000 |
| a2 | 4.000 | 3.000 | 0.000 | -1.000 | 1.000 | 0.000 | 6.000 |
| s3 | 1.000 | 2.000 | 0.000 | 0.000 | 0.000 | 1.000 | 4.000 |
| Dev.Row | 7M-4 | 4M-1 | 0 | -M | 0 | 0 | z=-9M |
Entering variable: x1 Leaving variable: a1

--- Table 2 ---
| Basis | x1 | x2 | a1 | s2 | a2 | s3 | RHS |
|---|---|---|---|---|---|---|---|
| x1 | 1.000 | 0.333 | 0.333 | 0.000 | 0.000 | 0.000 | 1.000 |
| a2 | 0.000 | 1.667 | -1.333 | -1.000 | 1.000 | 0.000 | 2.000 |
| s3 | 0.000 | 1.667 | -0.333 | 0.000 | 0.000 | 1.000 | 3.000 |
| Dev.Row | 0 | 1.6667M+0.3333 | 1.3333-2.3333M | -M | 0 | 0 | z=-4-2M |
Entering variable: x2 Leaving variable: a2

--- Table 3 ---
| Basis | x1 | x2 | a1 | s2 | a2 | s3 | RHS |
|---|---|---|---|---|---|---|---|
| x1 | 1.000 | 0.000 | 0.000 | 0.200 | -0.200 | 0.000 | 0.600 |
| x2 | 0.000 | 1.000 | -0.000 | -0.600 | 0.600 | 0.000 | 1.200 |
| s3 | 0.000 | 0.000 | 1.000 | 1.000 | -1.000 | 1.000 | 1.000 |
| Dev.Row | 0 | 0 | 1.6-M | 0.2 | -0.2-M | 0 | z=-3.6 |
Entering variable: s2 Leaving variable: s3

--- Table 4 ---
| Basis | x1 | x2 | a1 | s2 | a2 | s3 | RHS |
|---|---|---|---|---|---|---|---|
| x1 | 1.000 | 0.000 | 0.400 | 0.000 | 0.000 | -0.200 | 0.400 |
| x2 | 0.000 | 1.000 | -0.200 | 0.000 | 0.000 | 0.600 | 1.000 |
| s2 | 0.000 | 0.000 | 1.000 | 1.000 | -1.000 | 1.000 | 1.000 |
| Dev.Row | 0 | 0 | 1.4-M | 0 | -M | -0.2 | z=-3.4 |

No positive entries left in the Dev.Row -> optimal solution reached.

----- Final Answer -----
x1 = 0.4
x2 = 1.8
Optimal Z (minimum) = 3.4
```

1.6 Result

The simplex iterations converge in 4 tableaus. Both artificial variables leave the basis, confirming the problem is feasible, and the Dev.Row has no positive entries left at Table 4. The optimal solution obtained is **x1 = 0.4, x2 = 1.8**, giving **minimum Z = 3.4**.

Assignment 2 — Transportation Problem (VAM + MODI)

2.1 Objective

To write a Python program that finds the minimum-cost transportation plan for any number of sources and destinations, by first constructing an initial basic feasible solution with Vogel's Approximation Method (VAM) and then improving it to optimality with the Modified Distribution (MODI) method.

2.2 Theory in brief

VAM computes, for every row and column still active, a penalty equal to the difference between the lowest and second-lowest cost in it. The row or column with the largest penalty is selected, and as much as possible is allocated to its cheapest cell; whichever supply or demand is exhausted is then crossed out, and penalties are recomputed. This repeats until the whole supply/demand table is filled, giving a good starting solution.

MODI then finds dual values u_i (for each source) and v_j (for each destination) such that $u_i + v_j = c_{ij}$ for every occupied (basic) cell, starting from the row or column with the most allocations. For every empty cell, the net evaluation is $\Delta_{ij} = c_{ij} - (u_i + v_j)$. If every Δ_{ij} is ≥ 0 the solution is optimal (uniquely optimal if all are strictly positive); otherwise, the most negative cell is brought into the basis by tracing a closed loop through occupied cells and shifting the smallest allocation on a minus-corner of the loop around it. This repeats until every Δ_{ij} is ≥ 0 . A degeneracy check adds a near-zero allocation to an empty, loop-free cell whenever the basis has fewer than $m + n - 1$ occupied cells, since MODI needs a full basis to compute u_i and v_j .

2.3 Program design

get_problem_size_and_cost() reads the supply, demand and cost matrix, and adds a zero-cost dummy source or destination if total supply and total demand don't match, so the problem is always balanced before VAM runs.

vogel_approximation() implements the VAM loop described above, printing which row/column and cell is chosen at every round.

fix_degeneracy() and **find_closed_loop()** support MODI: the former tops up the basis to $m + n - 1$ occupied cells when needed, and the latter does a depth-first search for the closed loop of occupied cells (alternating row/column moves) used both for the degeneracy check and for the actual re-allocation step.

find_uv() solves for the dual values, and **modi_optimize()** runs the iterate-until-optimal loop, printing the u/v values and the Δ_{ij} table at every iteration before deciding whether to stop or shift allocations around the loop for the most negative cell.

2.4 Source code

```
1 import copy
2
3 TOL = 1e-6
4 EPS = 1e-3
5
6 def print_bordered(headers, rows, label_w=10, col_w=10):
7     def border():
8         return "+" * (label_w + 2) + ("+" + "-" * (col_w + 2)) * (len(headers) - 1) + "+"
9
10    def row_line(cells):
11        first = f" {str(cells[0]).center(label_w)} |"
12        rest = "".join(f" {str(c).center(col_w)} |" for c in cells[1:])
13        return "|" + first + rest
14
15    print(border())
16    print(row_line(headers))
17    print(border())
18    for r in rows:
19        print(row_line(r))
20    print(border())
21
22
23 def get_problem_size_and_cost():
24     m = int(input("How many sources (supply points)? "))
25     n = int(input("How many destinations (demand points)? "))
26
27     print(f"Enter the (m) supply values, space separated:")
28     supply = list(map(float, input("Supply: ").split()))
29
30     print(f"Enter the (n) demand values, space separated:")
31     demand = list(map(float, input("Demand: ").split()))
32
33     print(f"Now enter the cost matrix, one row at a time")
34     cost = []
35     for i in range(m):
36         row = list(map(float, input(f"Costs for source {i+1} ({n} values): ").split()))
37         cost.append(row)
38
39     total_supply = sum(supply)
40     total_demand = sum(demand)
41     if abs(total_supply - total_demand) > TOL:
42         print(f"Supply ({total_supply}) and demand ({total_demand}) don't match, "
43               f"adding a dummy 'destination' if total_supply > total_demand else 'source' "
44               f"with cost 0 to balance it.")
45     if total_supply > total_demand:
46         demand.append(total_supply - total_demand)
47         for row in cost:
48             row.append(0.0)
49     else:
50         supply.append(total_demand - total_supply)
51         cost.append([0.0] * n)
52
53     return supply, demand, cost
54
55 def vogel_approximation(cost, supply, demand):
56     rows, cols = len(supply), len(demand)
57     supply = supply[:]
58     demand = demand[:]
59     allocation = [[0.0] * cols for _ in range(rows)]
60     done_rows = set()
61     done_cols = set()
62     round_no = 0
63
64     while len(done_rows) < rows and len(done_cols) < cols:
65         round_no += 1
66         row_penalty = {}
67         for r in range(rows):
68             if r in done_rows:
69                 continue
70             vals = sorted(cost[r][c] for c in range(cols) if c not in done_cols)
71             row_penalty[r] = vals[1] - vals[0] if len(vals) > 1 else vals[0]
72
73         col_penalty = {}
74         for c in range(cols):
75             if c in done_cols:
76                 continue
77             vals = sorted(cost[r][c] for r in range(rows) if r not in done_rows)
78             col_penalty[c] = vals[1] - vals[0] if len(vals) > 1 else vals[0]
79
80         best_row = max(row_penalty, key=row_penalty.get) if row_penalty else None
81         best_col = max(col_penalty, key=col_penalty.get) if col_penalty else None
82
83         if best_row is not None and (best_col is None or row_penalty[best_row] >= col_penalty[best_col]):
84             r = best_row
85             c = min(c for c in range(cols) if c not in done_cols, key=lambda c: cost[r][c])
86             chosen = f"row {r+1} (penalty {row_penalty[best_row]:g})"
87         else:
88             c = best_col
89             r = min(r for r in range(rows) if r not in done_rows, key=lambda r: cost[r][c])
90             chosen = f"column {c+1} (penalty {col_penalty[best_col]:g})"
91
92         qty = min(supply[r], demand[c])
93         allocation[r][c] = qty
94         supply[r] -= qty
95         demand[c] -= qty
96
97         print(f"VAM round {round_no}: biggest penalty is {chosen} -> "
98               f"cheapest cell there is {S(r+1), D(c+1)}, allocate {qty:g}")
99
100        if supply[r] <= TOL:
101            done_rows.add(r)
102        if demand[c] <= TOL:
103            done_cols.add(c)
104
105    return allocation
106
107
108 def print_allocation(allocation, cost, supply_labels, demand_labels, title):
109     rows, cols = len(allocation), len(allocation[0])
110     print(f"({title})")
111     headers = [""] + [f"D{c+1}" for c in range(cols)]
112     table_rows = []
113     for r in range(rows):
114         cells = []
115         for c in range(cols):
116             if allocation[r][c] > TOL:
117                 cells.append(f"{allocation[r][c]:g} ({cost[r][c]:g})")
118             else:
119                 cells.append(f"--- ({cost[r][c]:g})")
120         table_rows.append([f"S{r+1}" + cells])
121     print_bordered(headers, table_rows, label_w=8, col_w=16)
122
123 def get_basic_cells(allocation):
124     rows, cols = len(allocation), len(allocation[0])
125     return [(r, c) for r in range(rows) for c in range(cols) if allocation[r][c] > TOL]
126
127
128 def fix_degeneracy(allocation, cost):
129     rows, cols = len(allocation), len(allocation[0])
130     needed = rows + cols - 1
131
132     """
133     """
```

```

133 while len(get_basic_cells(allocation)) < needed:
134     basic = set(get_basic_cells(allocation))
135     placed = False
136     for r in range(rows):
137         for c in range(cols):
138             if (r, c) in basic:
139                 continue
140             trial = set(basic)
141             trial.add((r, c))
142             if find_closed_loop(list(trial), (r, c)) is None:
143                 allocation[r][c] = EPS
144                 placed = True
145                 break
146             if placed:
147                 break
148             if not placed:
149                 break
150     return allocation
151
152
153 def find_uv(basic_cells, cost, rows, cols):
154     u = [None] * rows
155     v = [None] * cols
156
157     row_counts = [sum(1 for (r, c) in basic_cells if r == i) for i in range(rows)]
158     col_counts = [sum(1 for (r, c) in basic_cells if c == j) for j in range(cols)]
159     if max(row_counts) >= max(col_counts):
160         u[row_counts.index(max(row_counts))] = 0
161     else:
162         v[col_counts.index(max(col_counts))] = 0
163
164     changed = True
165     while changed:
166         changed = False
167         for (r, c) in basic_cells:
168             if u[r] is not None and v[c] is None:
169                 v[c] = cost[r][c] - u[r]
170                 changed = True
171             elif v[c] is not None and u[r] is None:
172                 u[r] = cost[r][c] - v[c]
173                 changed = True
174
175     u = [x if x is not None else 0 for x in u]
176     v = [x if x is not None else 0 for x in v]
177     return u, v
178
179
180 def next_loop_nodes(loop, candidates):
181     last = loop[-1]
182     if len(loop) < 2:
183         return [n for n in candidates if n[0] == last[0] or n[1] == last[1]]
184     prev = loop[-2]
185     if prev[0] == last[0]:
186         return [n for n in candidates if n[1] == last[1]]
187     else:
188         return [n for n in candidates if n[0] == last[0]]
189
190
191 def find_closed_loop(basic_cells, start):
192     def search(loop):
193         if len(loop) >= 4 and start in next_loop_nodes(loop, [start]):
194             return loop
195         remaining = [cell for cell in basic_cells if cell not in loop]
196         for nxt in next_loop_nodes(loop, remaining):
197             result = search(loop + [nxt])
198             if result:
199                 return result
200         return None
201     return search([start])
202
203
204 def mod1_optimize(allocation, cost):
205     allocation = copy.deepcopy(allocation)
206     rows, cols = len(allocation), len(allocation[0])
207     allocation = fix_degeneracy(allocation, cost)
208
209     iteration = 0
210     while True:
211         iteration += 1
212         basic_cells = get_basic_cells(allocation)
213         u, v = find_uv(basic_cells, cost, rows, cols)
214
215         print(f"\n--- MOD1 iteration {iteration} ---")
216         print("u values:", [round(x, 3) for x in u])
217         print("v values:", [round(x, 3) for x in v])
218
219         deltas = {}
220         for r in range(rows):
221             for c in range(cols):
222                 if allocation[r][c] <= TOL:
223                     deltas[(r, c)] = cost[r][c] - (u[r] + v[c])
224
225         headers = [""] + [f"D{c+1}" for c in range(cols)]
226         table_rows = []
227         for r in range(rows):
228             cells = []
229             for c in range(cols):
230                 if allocation[r][c] > TOL:
231                     cells.append(f"{allocation[r][c]:g}")
232                 else:
233                     cells.append(f"D={deltas[(r, c)]:.2f}")
234             table_rows.append([f"S{r+1}" + cells])
235         print_bordered(headers, table_rows, label_w=8, col_w=12)
236
237         if not deltas or all(d >= -TOL for d in deltas.values()):
238             if all(d > TOL for d in deltas.values()):
239                 print("All Δij > 0 -> optimal, and this is the unique optimum.")
240             else:
241                 print("All Δij >= 0 -> optimal, but an alternate optimal solution also exists.")
242             break
243
244         entering_cell = min(deltas, key=deltas.get)
245         print(f"Most negative Δij is at ({entering_cell[0]+1}, {entering_cell[1]+1})" +
246               f" -> not optimal yet, bringing this cell in.")
247
248         loop = find_closed_loop(basic_cells, entering_cell)
249         minus_cells = loop[1::2]
250         theta = min(allocation[r][c] for (r, c) in minus_cells)
251
252         for i, (r, c) in enumerate(loop):
253             if i % 2 == 0:
254                 allocation[r][c] += theta
255             else:
256                 allocation[r][c] -= theta
257
258     for r in range(rows):
259         for c in range(cols):
260             if allocation[r][c] < 1e-2:
261                 allocation[r][c] = 0.0
262
263     return allocation
264
265
266 if __name__ == "__main__":
267     print("==== Transportation Problem Solver (VAM + MOD1) ====")
268
269     print("\n--- Input Data ---")
270     supply, demand, cost = get_problem_size_and_cost()
271
272     print("\n--- Phase 1: Vogel's Approximation Method (VAM) ---")
273     print("Distributing units row by row / column by column until every")
274     print("supply and demand value is used up, to build the baseline plan.\n")
275     allocation = vogel_approximation(cost, supply, demand)
276     print_allocation(allocation, cost, supply, demand, "\nBaseline plan from VAM:")
277     vam_cost = sum(allocation[r][c] * cost[r][c] for r in range(len(cost)) for c in range(len(cost[0])))
278     print(f"\nCost of the VAM baseline plan: {vam_cost:g}")
279
280     print("\n--- Phase 2: MOD1 (Modified Distribution) Method ---")
281     print("Computing u_i and v_j for every occupied cell, checking each empty")
282     print("cell's Delta_ij, and looping the allocation around whenever a cell")
283     print("shows the cost can still be reduced.\n")
284     final_allocation = mod1_optimize(allocation, cost)
285     print_allocation(final_allocation, cost, supply, demand, "\nFinal (optimal) allocation after MOD1:")
286     final_cost = sum(final_allocation[r][c] * cost[r][c] for r in range(len(cost)) for c in range(len(cost[0])))
287     print(f"\nMinimum total transportation cost: {final_cost:g}")

```

2.5 Sample run

The program was tested on a classic balanced transportation problem with 3 sources and 4 destinations:

	D1	D2	D3	D4	Supply
S1	3	1	7	4	300
S2	2	6	5	9	400
S3	8	3	3	2	500
Demand	250	350	400	200	1200

Console input/output :

```
=== Transportation Problem Solver (VAM + MODI) ===

--- Input Data ---
How many sources (supply points)? 3
How many destinations (demand points)? 4

Enter the 3 supply values, space separated:
Supply: 300 400 500

Enter the 4 demand values, space separated:
Demand: 250 350 400 200

Now enter the cost matrix, one row at a time
Costs for source 1 (4 values): 3 1 7 4
Costs for source 2 (4 values): 2 6 5 9
Costs for source 3 (4 values): 8 3 3 2

--- Phase 1: Vogel's Approximation Method (VAM) ---
Distributing units row by row / column by column until every
supply and demand value is used up, to build the baseline plan.

VAM round 1: biggest penalty is row S2 (penalty 3) -> cheapest cell there is (S2, D1), allocate 250
VAM round 2: biggest penalty is row S1 (penalty 3) -> cheapest cell there is (S1, D2), allocate 300
VAM round 3: biggest penalty is column D4 (penalty 7) -> cheapest cell there is (S3, D4), allocate 200
VAM round 4: biggest penalty is column D2 (penalty 3) -> cheapest cell there is (S3, D2), allocate 50
VAM round 5: biggest penalty is row S2 (penalty 5) -> cheapest cell there is (S2, D3), allocate 150
VAM round 6: biggest penalty is row S3 (penalty 3) -> cheapest cell there is (S3, D3), allocate 250

Baseline plan from VAM:
+-----+-----+-----+-----+
|       | D1    | D2    | D3    | D4    |
+-----+-----+-----+-----+
| S1     | - (c=3) | 300 (c=1) | - (c=7) | - (c=4) |
| S2     | 250 (c=2) | - (c=6) | 150 (c=5) | - (c=9) |
| S3     | - (c=8) | 50 (c=3) | 250 (c=3) | 200 (c=2) |
+-----+-----+-----+-----+

Cost of the VAM baseline plan: 2850

--- Phase 2: MODI (Modified Distribution) Method ---
Computing u_i and v_j for every occupied cell, checking each empty
cell's Delta_ij, and looping the allocation around whenever a cell
shows the cost can still be reduced.

--- MODI iteration 1 ---
u values: [-2.0, 2.0, 0]
v values: [0.0, 3.0, 3.0, 2.0]
+-----+-----+-----+-----+
|       | D1    | D2    | D3    | D4    |
+-----+-----+-----+-----+
| S1     | D=5.00 | 300    | D=6.00 | D=4.00 |
| S2     | 250    | D=1.00 | 150    | D=5.00 |
| S3     | D=8.00 | 50     | 250    | 200    |
+-----+-----+-----+-----+

All Delta_ij > 0 -> optimal, and this is the unique optimum.

Final (optimal) allocation after MODI:
+-----+-----+-----+-----+
|       | D1    | D2    | D3    | D4    |
+-----+-----+-----+-----+
| S1     | - (c=3) | 300 (c=1) | - (c=7) | - (c=4) |
| S2     | 250 (c=2) | - (c=6) | 150 (c=5) | - (c=9) |
| S3     | - (c=8) | 50 (c=3) | 250 (c=3) | 200 (c=2) |
+-----+-----+-----+-----+

Minimum total transportation cost: 2850
```

2.6 Result

VAM produces an initial feasible plan with cost 2850. Running MODI on it, all Delta_ij values for the empty cells come out strictly positive on the very first iteration, so the VAM solution is already optimal (and uniquely so). The **minimum total transportation cost is 2850**, achieved by shipping S1→D2: 300, S2→D1: 250, S2→D3: 150, S3→D2: 50, S3→D3: 250, S3→D4: 200.